

Задание 1

Что нужно сделать:

1. Разверните GitLab локально, используя Vagrantfile и инструкцию, описанные в [этом репозитории](#).
2. Создайте новый проект и пустой репозиторий в нём.
3. Зарегистрируйте gitlab-runner для этого проекта и запустите его в режиме Docker. Раннер можно регистрировать и запускать на той же виртуальной машине, на которой запущен GitLab.

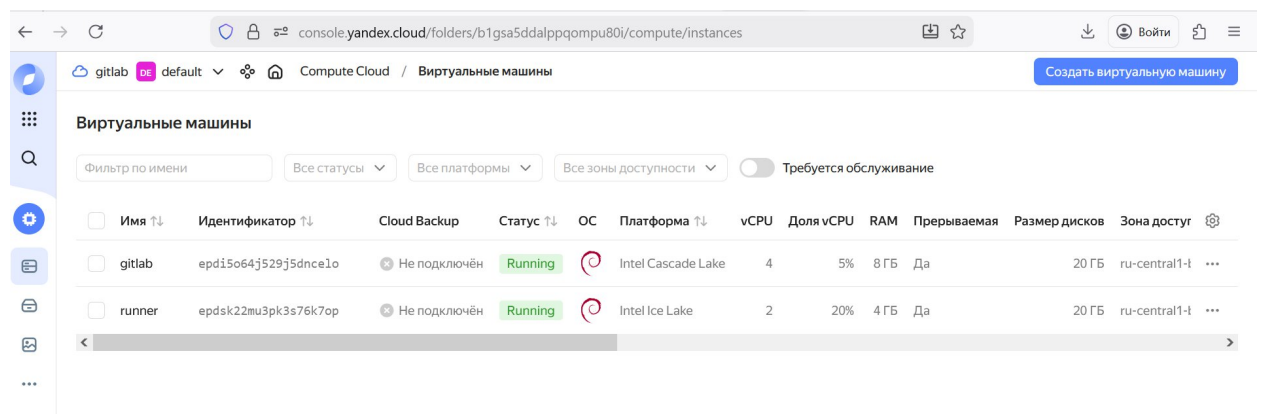
В качестве ответа в репозиторий шаблона с решением добавьте скриншоты с настройками раннера в проекте.

Ответ:

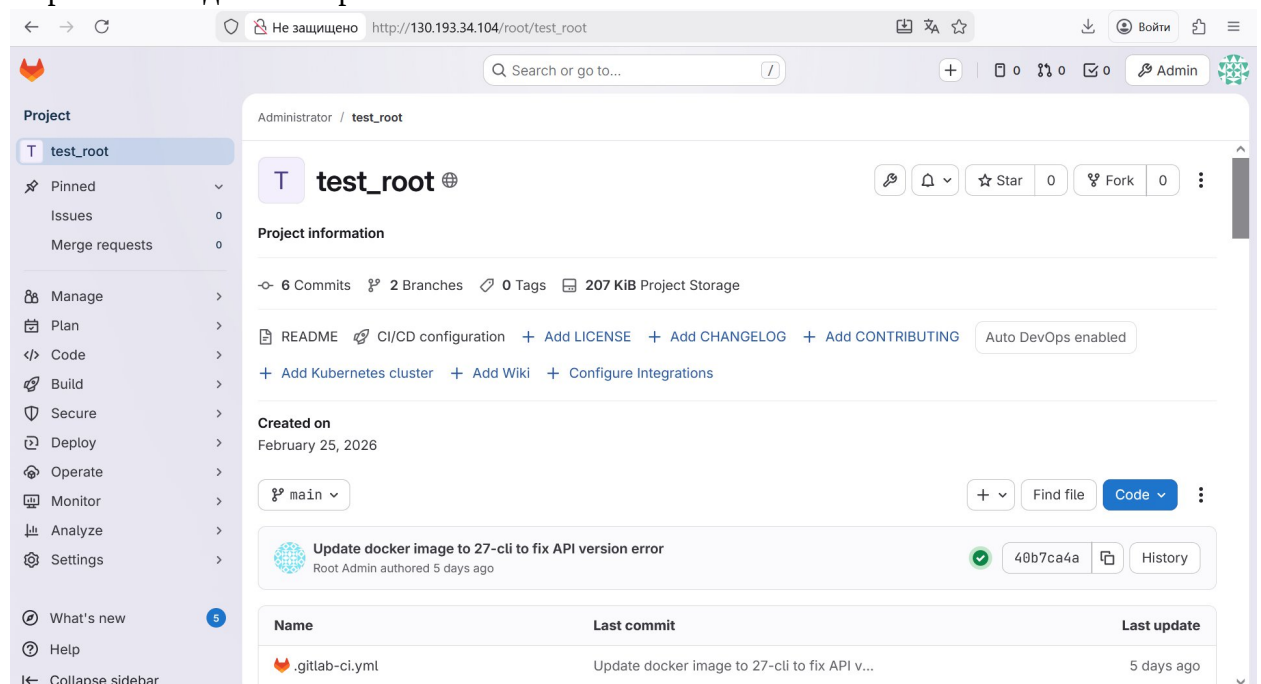
Локальный GitLab разворачивался без использования Vagrantfile в YandexCloud.

Runner развернут на отдельной виртуальной машине в YandexCloud.

Связь между локальным Gitlab и Runner по внутренним адресам внутри YandexCloud.



Скриншот созданного проекта:



Настройки Runner:

Настройки находятся в файле в /srv/gitlab-runner/config/config.toml

```
1 concurrent = 1
2 check_interval = 0
3 shutdown_timeout = 0
4
5 [session_server]
6   session_timeout = 1800
7
8 [[runners]]
9   name = "57e1b8dc7e46"
10  url = "http://10.129.0.14"
11  id = 3
12  token = "glrt-c_3xXBt-5gPDm9-RUahWtG86MQp00jEKdT0xCw.01.121kv95mt"
13  token_obtained_at = 2026-02-25T13:28:40Z
14  token_expires_at = 0001-01-01T00:00:00Z
15  executor = "docker"
16  environment = ["DOCKER_TLS_CERTDIR="]
17  [runners.cache]
18    MaxUploadedArchiveSize = 0
19    [runners.cache.s3]
20    [runners.cache.gcs]
21    [runners.cache.azure]
22  [runners.docker]
23    tls_verify = false
24    image = "ruby:2.7"
25    privileged = true
26    disable_entrypoint_overwrite = false
27    oom_kill_disable = false
28    disable_cache = false
29    volumes = ["/cache", "/var/run/docker.sock:/var/run/docker.sock"]
30    shm_size = 0
31    network_mtu = 0
32    extra_hosts = ["gitlab.example.com:10.129.0.14", "localhost:10.129.0.14"]
33
```

Задание 2

Что нужно сделать:

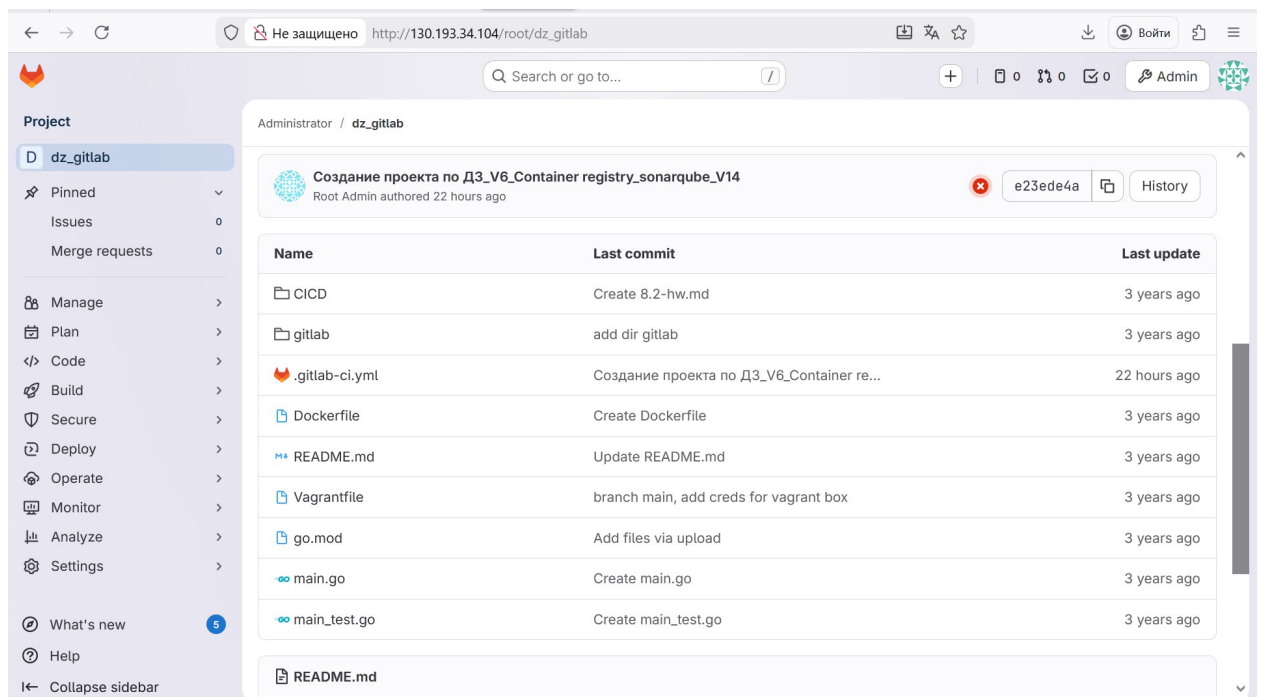
1. Запустите [репозиторий](#) на GitLab, изменив origin. Это изучалось на занятии по Git.
2. Создайте `.gitlab-ci.yml`, описав в нём все необходимые, на ваш взгляд, этапы.

В качестве ответа в шаблон с решением добавьте:

- файл `gitlab-ci.yml` для своего проекта или вставьте код в соответствующее поле в шаблоне;
- скриншоты с успешно собранными сборками.

Ответ:

Скриншот запущенного проекта с GitLab Нетодологии в мой локальный GitLAB:



Конфиг файла `gitlab-ci.yml`:

```
image: docker:latest

services:
  - docker:dind

variables:
  # Адрес реестра и имя образа
  IMAGE_NAME: $CI_REGISTRY_IMAGE:latest

stages:
  - build
  - test

build_app:
  stage: build
```

script:

- # 1. Логинимся в Registry под системным пользователем GitLab
- docker login -u \$CI_REGISTRY_USER -p \$CI_REGISTRY_PASSWORD \$CI_REGISTRY
- # 2. Собираем образ с тегом
- docker build -t \$IMAGE_NAME .
- # 3. Отправляем (пушим) в Container Registry на моем локальном gitlab
- docker push \$IMAGE_NAME

tags:

- docker

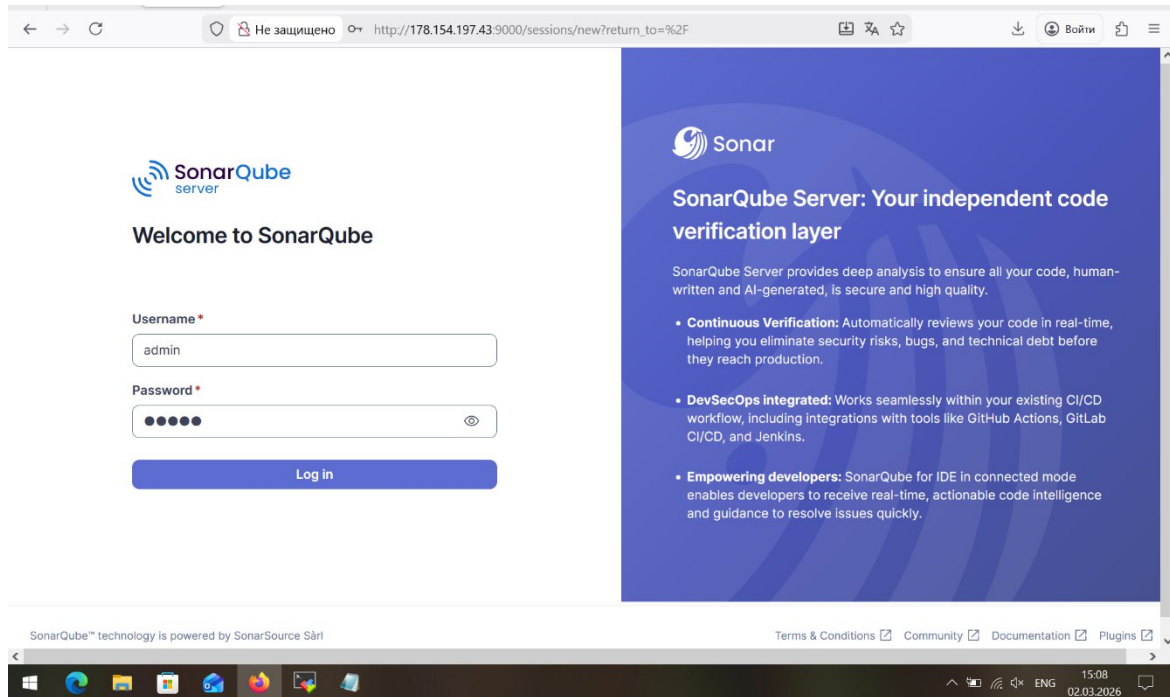
test_infra:

stage: test

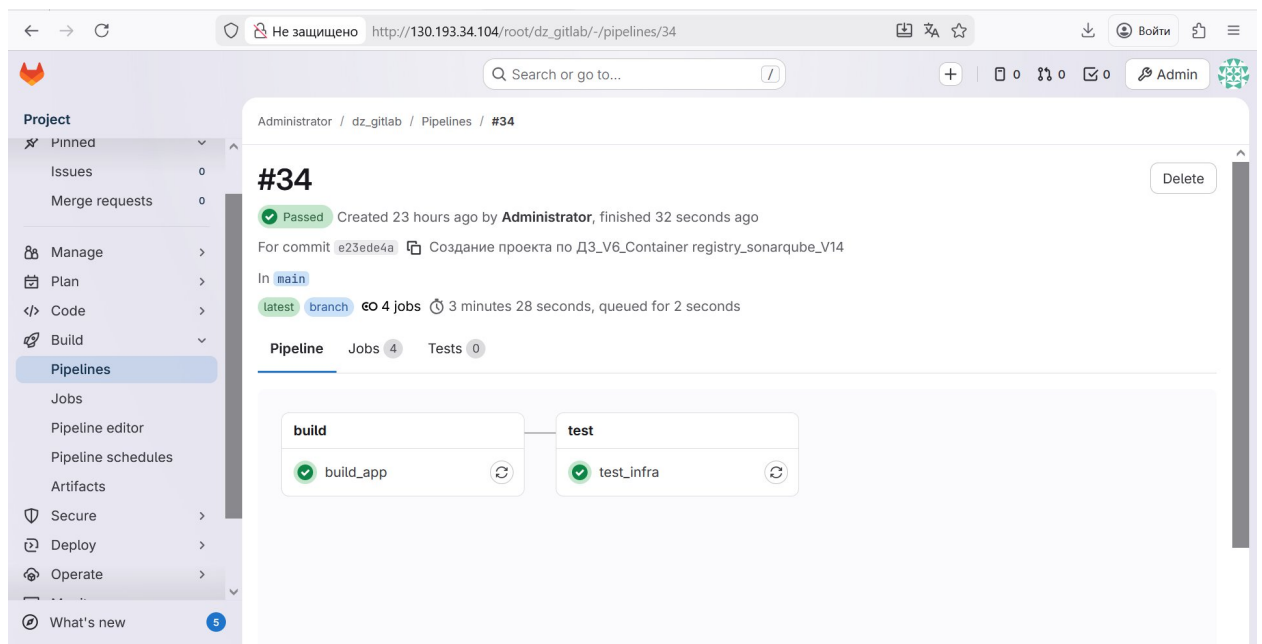
script:

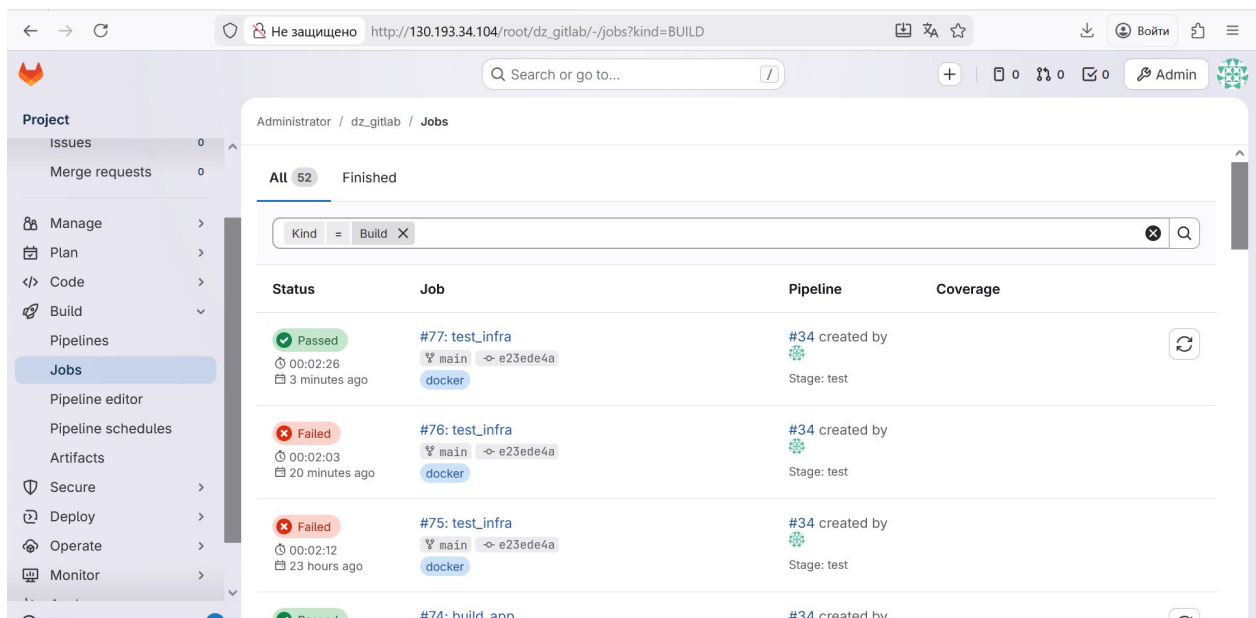
- cd gitlab/
 - docker compose up -d
 - echo "Waiting for SonarQube..."
 - sleep 60
 - # ПОСМОТРИМ ОШИБКУ: выводим последние 50 строк логов именно сонара
 - docker compose logs sonarqube | | echo "Не удалось получить логи"
 - docker compose ps
 - docker compose logs sonarqube
 - # Находим ID контейнера, который слушает порт 9000, и берем его IP
 - SONAR_IP=\$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
 - \$(docker ps -q --filter "publish=9000"))
 - echo "Найденный IP SonarQube - \$SONAR_IP"
 - # Запускаем сканер через общую сеть
 - docker run --rm --network gitlab_default -e SONAR_HOST_URL="http://\$SONAR_IP:9000" -v
 - "\$PWD:/usr/src" sonarsource/sonar-scanner-cli -Dsonar.projectKey=dz_gitlab -
 - Dsonar.token=squ_a19cb3c86ed6815aa7d12cb393bcab5d4aef8448
 - docker compose down
- tags:
- docker

Скриншот рабочего SonarQube:



Скриншот сборки:





С какими проблемами пришлось столкнуться:

1. Для корректного запуска SonarQube на хост-машине был увеличен лимит областей виртуальной памяти `vm.max_map_count` до 262144, что позволило избежать ошибки запуска Elasticsearch.
2. Для передачи результатов анализа в SonarQube был сгенерирован и использован Personal Access Token (ключ). Он передан сканеру через параметр `-Dsonar.token`, что устранило ошибку 401 Unauthorized и позволило успешно произвести проверку кода на GOLANG.
3. В этапе `test_infra` реализован цикл:
 - Развертывание стека (SonarQube + PostgreSQL) через `docker compose`.
 - Ожидание готовности сервиса (`sleep`).
 - Статический анализ кода через `sonar-scanner-cli`.
 - Очистка ресурсов: Все временные контейнеры и сети удаляются командой `docker compose down` сразу после завершения задачи.

Задание 3*

Измените CI так, чтобы:

- этап сборки запускался сразу, не дожидаясь результатов тестов;
- тесты запускались только при изменении файлов с расширением *.go.

В качестве ответа добавьте в шаблон с решением файл gitlab-ci.yml своего проекта или вставьте код в соответствующее поле в шаблоне.

Ответ:

Для выполнения этого задания в GitLab CI используются два ключевых механизма:

needs (для нарушения очередности стадий) и **rules:changes** (для фильтрации запуска по расширению файлов).

В результате gitlab-ci.yml будет иметь следующий вид:

```
image: docker:latest

services:
  - docker:dind

variables:
  # Адрес реестра и имя образа
  IMAGE_NAME: $CI_REGISTRY_IMAGE:latest

stages:
  - build
  - test

build_app:
  stage: build
  # Заставляем сборку запускаться сразу, не дожидаясь других задач
  needs: []
  script:
    # 1. Логинимся в Registry под системным пользователем GitLab
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    # 2. Собираем образ с тегом
    - docker build -t $IMAGE_NAME .
    # 3. Отправляем (пушим) в Container Registry на моем локальном gitlab
    - docker push $IMAGE_NAME
  tags:
    - docker

test_infra:
  stage: test
  # Запускать тесты только если изменились файлы .go
  rules:
    - changes
      - "**/*.go"
  script:
```

```

- cd gitlab/
- docker compose up -d
- echo "Waiting for SonarQube...Ожидаем загрузки SonarQube"
- sleep 60
# ПОСМОТРИМ ОШИБКУ: выводим последние 50 строк логов именно сонара
- docker compose logs sonarqube || echo "Не удалось получить логи"
- docker compose ps
- docker compose logs sonarqube
# Находим ID контейнера, который слушает порт 9000, и берем его IP
- SONAR_IP=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
$(docker ps -q --filter "publish=9000"))
- echo "Найденный IP SonarQube - $SONAR_IP"
# Запускаем сканер через общую сеть
- docker run --rm --network gitlab_default -e SONAR_HOST_URL="http://$SONAR_IP:9000" -v
"$PWD:/usr/src" sonarsource/sonar-scanner-cli -Dsonar.projectKey=dz_gitlab -
Dsonar.token=squ_a19cb3c86ed6815aa7d12cb393bcab5d4aef8448
- docker compose down
tags:
- docker

```

Проверка:

Тест №1 Добавлю пару пустых строчек в файле gitlab-ci.yml для теста.

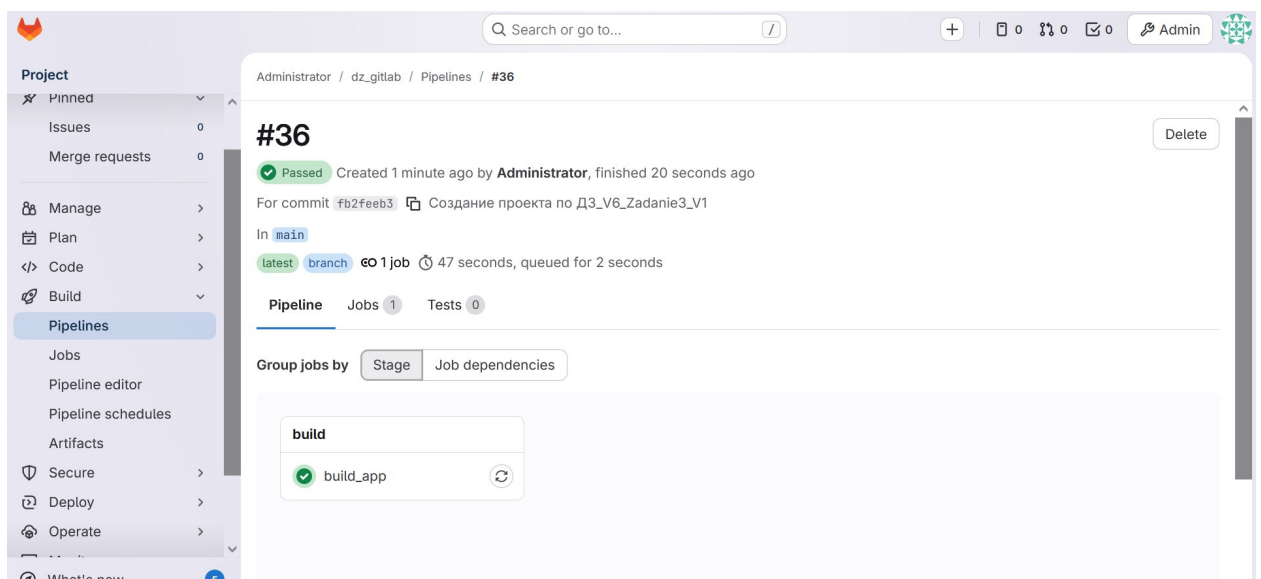
Сделаю:

```
git add .gitlab-ci.yml
```

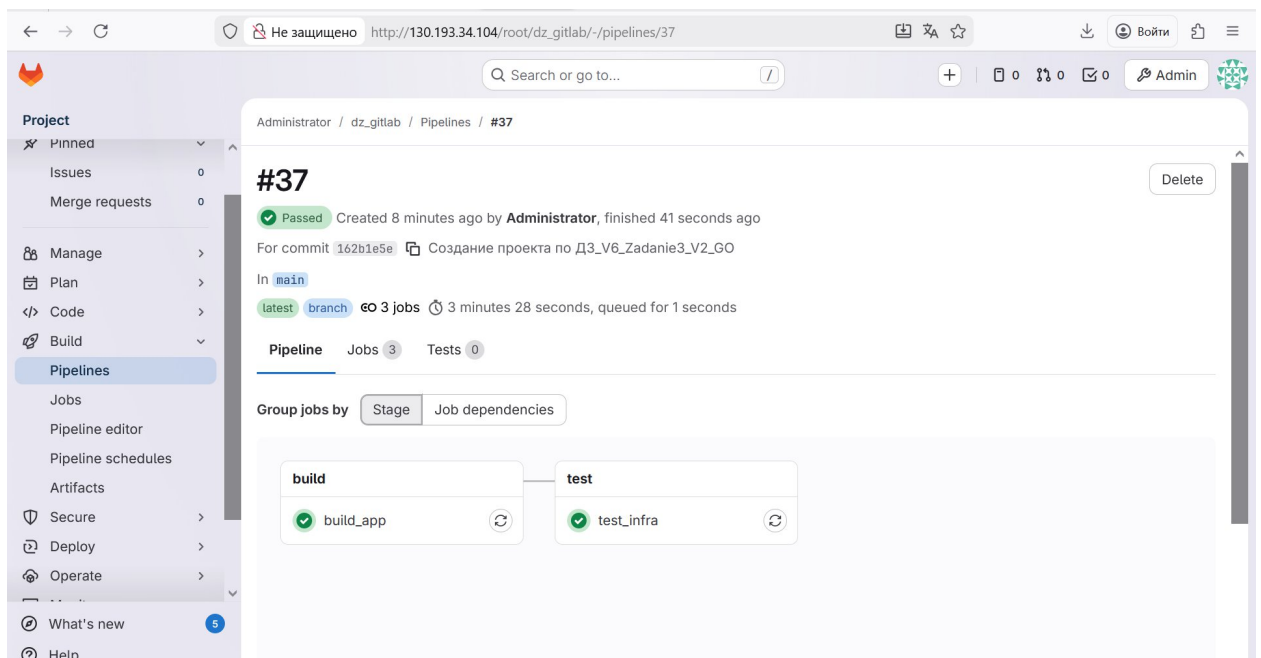
```
git commit -m "Создание проекта по ДЗ_V6_Zadanie3_V1"
```

```
git push
```

После данных команд и изменения файла в Pipeline появилась только задача build_app. Стадия test отсутствует – все работает, (файл .go не менялся при тестировании).



Тест №2 Произвожу изменения в файла с расширением .go, после git commit и git push в Pipeline появились обе задачи: и build_app и test_infra



Дополнительно я задействовал Container Registry, мои контейнеры с проектом появляются при каждом запуске пайплайна.

Наименование контейнеров меняется благодаря изменению в конфигурации файла gitlab-ci.yml

```
variables:  
# Адрес реестра и имя образа  
IMAGE_NAME: $CI_REGISTRY_IMAGE:$CI_PIPELINE_IID
```

